Ders 8 Genişletilmiş Notu: Nesneye Dayalı Tasarım: Etkileşim Diyagramları, GRASP Sorumluluk Atama Prensipleri ve Tasarım Sınıf Diyagramı

Ders 8 Genişletilmiş Notu: Nesneye Dayalı Tasarım: Etkileşim Diyagramları, GRASP Sorumluluk Atama Prensipleri ve Tasarım Sınıf Diyagramı

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Nerede Kalmıştık: Tasarımın Yeri
- 2. Nesneye Dayalı Tasarım Modeli
- 3. Etkileşim Diyagramları
- 4. Kullanım Senaryolarının Gerçeklenmesi
- 5. Satış Örneği ve GRASP'a Giriş
- 6. Uzman (Expert)
- 7. Yaratıcı (Creator)
- 8. Az Bağımlılık (Low Coupling)
- 9. İyi Uyum (High Cohesion)
- 10. Denetçi (Controller)
- 11. Çok Şekillilik (Polymorphism)
- 12. Yapay Sınıf (Pure Fabrication)
- 13. Dolaylılık ve Ara Birim (Indirection)
- 14. Değişimlerden Korunma (Protected Variations)
- 15. Yabancılarla Konuşma Prensibi
- 16. İlklendirme Senaryosu
- 17. Görünürlük (Visibility)
- 18. Tasarım Sınıf Diyagramı
- 19. Kodlamaya Geçiş

1. Nerede Kalmıştık: Tasarımın Yeri

Bu derste bir bilgi sisteminin ortaya çıkarılma sürecinde hangi adımların olduğunu ve her adımda ne yapıldığını öğreniyoruz. Sürece ön inceleme ve fizibilite çalışmasıyla başlamış, ardından iki alt aşamadan oluşan analizi görmüştük. Şimdi uygulama ve yazılım tasarımının ayrıntılarındayız. Geçen hafta başladığımız bu konu, bu hafta nesneye yönelik tarafıyla tamamlanıyor.

Tasarımı da iki alt parçaya ayırmıştık:

- **Ön tasarım:** Eski sistemden neyi ne kadar kurtarabileceğimize, neyi ne kadar hazır kullanabileceğimize ve bunları ne kadar değiştirmemiz gerektiğine bakarız. Fizibiliteden gelen tekliflerin ne kadarının gerçeğe dönüştürülebileceğini ve nasıl değişiklikler gerektiğini belirleriz. Ana modüler çözümlemeyi de burada ortaya koyarız.
- **Ayrıntılı tasarım:** Kullanıcılarla, dış sistemlerle ve bütün paydaşlarla etkileşimi sağlayan girdi, çıktı, ara birim ve arayüz tasarımları burada yapılır. Program tasarımı, yani

yazılımın tasarlanması ve veri yapılarının ya da onların duracağı veri tabanının tasarımı da bu aşamadadır.

Geçen hafta işlevsel tarafta modüler çözümlemeyi ele almıştık. Veri akış diyagramlarından yola çıktıysanız modüler çözümlemeyi yapı diyagramlarıyla anlatmak gerekir. Fiziksel veri akış diyagramları, hatta onların içindeki süreç akış formları da yeterli olabilir. Bunlar yetmiyorsa, eskide kalmış olsa da hierarchical input process output, yani HIPO gösterimini kullanabilirsiniz. Her süreç için ayrı girdi-işlem-çıktı kartları ya da daha ayrıntılı kartlar oluşturursunuz. Böylece iş mantığının, yani algoritmaların nasıl olması gerektiği bu kartlarda yer alır.

Bu haftanın sorusu şudur: Nesneye yönelik çalışıyorsak, ayrıntılı tasarımın içindeki yazılım tasarımı nasıl yapılır? Çalışacak sistemin kodlarının nasıl olması gerektiğine dair çözüm önerisi nasıl oluşturulur, nasıl ifade edilir, hangi diyagramlar neden kullanılır?

Hoca kapsamı da netleştirdi: Nesneye yönelik çalışsanız da girdi, çıktı, ara birim, arayüz ve veri tabanı tasarımlarını yine yapacaksınız. Ama bu derste odak yalnızca yazılımın kendisidir, yani çalışacak uygulamanın nasıl tasarlanacağı ve bu tasarımın nasıl ifade edileceğidir.

2. Nesneye Dayalı Tasarım Modeli

2.1 “Ne” Sorusundan “Nasıl” Sorusuna

Analizde hep “ne?” sorusuna cevap vermiştik: Kavramlar neler, yapılması gerekenler neler, sorumluluklar neler? Tasarımda ise “nasıl?” sorusunu yanıtlıyoruz. Bir iş yapılacak, bir sorumluluk yerine getirilecek, bir gereksinim var. Bu nasıl yapılacak? Bununla ilgili mantıksal çözümü oluşturma aşamasına **tasarım modelinin oluşturulması** diyoruz.

Bu aşamada artık yazılım sınıflarını ortaya çıkarıyor ve bu sınıfların bir arada nasıl çalıştığını, yani nasıl etkileşimde bulunduğunu gösteriyoruz. Bunun için bu derste iki çeşit diyagram kullanılıyor. UML’de tasarımda ve analizde kullanılan çok daha fazla diyagram var. Hoca bu derste yalnızca işimizi görecek kadarını tanıttığını, ayrıntıların gerekirse uzmanlık gerektiren ileri seçimlik derslere kaldığını söyledi.

- **Tasarım sınıf diyagramı (design class diagram, software class diagram):** Analizde kavramsal sınıf diyagramı dediğimiz diyagram, tasarımda bu adı alır. Yazılımın statik yapısını, yani yapı parçalarını gösterir. Hoca iki adın da aynı şey olduğunu, genellikle “design class diagram” denildiğini belirtti.
- **Etkileşim diyagramları:** Bütün sınıfların bir arada nasıl çalışacağını, kimin kimden ne hizmet alıp vereceğini gösterir. Amaç sorumlulukların yerine getirilmesini ve senaryoların gerçeklenmesini ifade etmektir. İki türü vardır ve bunlar birbirinin alternatifidir.

Yaptığımız şey, bütün kavramsal sınıfları gerçek dünyaya mümkün olduğunca benzer biçimde yazılım uzayına taşımak, yazılım sınıfı hâline getirmek ve onlara davranışlar eklemektir. Etkileşim dediğimiz şey sınıfların birlikte davranmasıdır. Bu sayede yazılımda çalışacak sınıflara sorumluluklarını atamaya başlarız.

> **Hocanın vurgusu:** “Kim neden sorumlu olsun?” Tasarımda bütün olay bunun kararını vermektir. Hangi kavram neden sorumlu, hangi kavram ne hizmet vermeli, hangi kavramlar bir arada çalışarak bir senaryoyu yerine getirmeli?

2.2 Tasarımın Girdileri ve Akışı

Tasarım iki noktadan beslenir. Sorumluluklara ihtiyacımız vardır ve bunlar kullanım senaryolarında bulunur. Bir de sözleşmeler vardır. Sözleşmeler, kısa senaryoları ya da belli son durumların nasıl oluştuğunu daha ayrıntılı açıklamak için kullanılır. Hoca bu kısmı atladıklarını, burada da üzerinde durmayacağını söyledi. Senaryolardaki fiiller yapılması gereken işleri gösterir. Tasarımda bu işlerin nasıl yapılacağını ortaya koyarız.

Bunu yaparken belli prensipler ve daha da ötesi belli **tasarım kalıpları** kullanırız. Tasarım kalıbı, benzer problemlere insanların daha önce kafa yorarak ürettiği ve iyi çalıştığı bilinen çözümlerdir.

> **Hocanın vurgusu:** Tekerleği baştan icat etmiyoruz. Benzer bir problemde yapılmış ve iyi çalıştığı bilinen bir çözüm varsa, oradaki sınıf ve sorumluluk dağılımını aynen

kendi tasarımımıza taşıyoruz. Mümkün olduğunca böyle çalışmaya gayret ediyoruz.

Akış şöyledir:

1. Kullanım senaryolarından sorumluluklar çıkarılır.
2. Tasarım kalıpları ve prensipleri ışığında sorumluluklar ilgili sınıflara atanır. Asıl tasarım budur.
3. Sonuç, dinamik taraf için etkileşim diyagramlarıyla (ardışıl ya da işbirliği), statik taraf için tasarım sınıf diyagramıyla ifade edilir.

3. Etkileşim Diyagramları

Etkileşim diyagramları, bir sorumluluğun kavramlar bir arada çalışarak nasıl yerine getirildiğini gösterir. İki çeşidi vardır:

	İşbirliği diyagramı (collaboration diagram)	Ardışıl diyagram (sequence diagram)
Yerleşim	Düzlemsel, nesneler sanki bir masanın üstünde duruyor	Nesneler yan yana, her birinin yukarıdan aşağı yaşam ömrü var
Yer	Az yer kaplar	Daha fazla yer kaplar
Dallanma, döngü	Daha basit gösterilir	Kutularla gösterilir
Mesaj sırası	Takibi daha zor, numaralardan okunur	Çok daha kolay anlaşılır

İkisi de aynı bilgiyi verir. Birbirinin tam alternatifidir, hangisini kullandığınızın önemi yoktur. Ama mesaj sırası daha kolay anlaşıldığı için genellikle ardışıl diyagram kullanılır.

Hoca bir uyarı da yaptı: Ardışıl diyagramları analiz modelinde, özellikle aktör ve senaryo etkileşimlerinde kullanan çözümler de vardır. Bazı kaynaklarda bunu görürseniz şaşırmayın. Ama esas kullanım amacı tasarımda yazılım sınıflarının bir arada nasıl iş birliği yapacağını göstermektir.

> **Ders sonrası not:** Hoca "collaboration diagram" adını kullandı. Bu diyagramın UML 2'deki adı büyük olasılıkla iletişim diyagramıdır (communication diagram). Başka kaynaklarda bu adla karşılaşırsanız aynı diyagramı anlatır.

3.1 Nesne Gösterimi ve Temel Örnek

Nesneler dikdörtgenlerle gösterilir. Önce örnek adı (instance name), sonra iki nokta üst üste, sonra sınıf adı yazılır: `n1 : Nesne`. Bu, `Nesne` isimli sınıftan `n1` nesnesini türettiğimiz, yani "n1 = new Nesne" dediğimiz anlamına gelir. Hocanın örneğinde aynı türden dört nesne vardır: n1, n2, n3, n4.

İşbirliği diyagramında bir mesaj n1'e gelir, yani n1 üzerinde bir metot çağrısı yapılır. Bu metot işini yapabilmek için başka nesnelerle çalışır. Silsile numaralarla izlenir:

- **1:** n1'e gelen çağrı.
  - **1.1:** n1, n2'deki bir metodu çağırır.
  - **1.2:** n1, n3'teki bir metodu çağırır. Bu metot çalışırken n3, n2'deki başka bir metodu çağırır.
- **2:** Bu mesajlaşma bittikten sonra n1, n4'teki başka bir metodu çağırır.

Hocaya göre bütün bu çağrılar, n1'e gelen metodun yaptığı işin parçasıdır. Diyagramı çizdiğimizde şunu göstermiş oluruz: Yapılması gereken bir iş var, n1 bu işi yapabilmek için mesela üç ayrı nesneyle çalışmak zorunda. Bunu okuyabilmek için numaralara bakmak gerekir. Hocanın dediği gibi, sırayı takip etmek bu diyagramda biraz daha zordur.

Aynı senaryo ardışıl diyagramda şöyle görünür. Her nesnenin yukarıdan aşağı inen bir yaşam çizgisi vardır ve zaman yukarıdan aşağı akar. Çağrılan metotlar kalın çubuklar olarak çizilir: Çubuğun başladığı yerde metot çağrılmış, bittiği yerde çalışması tamamlanmış olur. n1'e mesaj gelince bir çubuk başlar. Onun içinden n2'ye bir çağrı çıkar. Ardından n3'e bir çağrı çıkar ve

n3'ün çubuğu çalışırken ondan n2'ye başka bir çağrı gider. Sonra n1'den n4'e çağrı yapılır. İç içe çağrılar, iç içe çubuklar olarak doğrudan görülür.

Ardışıl diyagramın başında bazen bir aktör de çizilir. Senaryoyu tetikleyen bir kullanıcı ilk nesnenin metodunu çağırır. Hocanın örneği: Ekranda "satışı tamamla" diye bir buton vardır, kullanıcı buna tıklar. Bu tıklamayla aslında "haydi satışın toplam tutarını hesapla" gibi bir iş başlatılmış olur.

3.2 Geri Dönüş Değeri

Bir metodun döndürdüğü değeri göstermek isterseniz, çağrılan metottan çağırana doğru **kesikli bir geri dönüş oku** çizer, üzerine dönen değeri yazarsınız. Hoca bunu ardışıl diyagram üzerinde canlı olarak çizdi. Normalde dönüş okları gösterilmez. Yalnızca dönen şeyin ne olduğunu özellikle söylemek istiyorsanız eklenir.

3.3 Kendine Mesaj

Nesneler başka nesnelerin metotlarını çağırabildiği gibi kendi içindeki başka bir metodu da çağırabilir. Bu, nesnenin "self nokta mesaj" çağırması gibidir. İşbirliği diyagramında nesneden kendisine dönen bir mesajla gösterilir. Ardışıl diyagramda ise çağrı, başka bir metodun gövdesinin içinde olmalıdır: n1'in çalışan bir metodu vardır ve bu gövdenin içinde yine n1'e ait başka bir metot çalışır. Çubuktan çıkıp aynı nesneye dönen okla gösterilir.

3.4 Koşullu Mesaj

Metot çağrısı bir koşula bağlı da olabilir. Koşul köşeli parantez içinde yazılır. Hocanın örneği: `[not ortalaması > 3]` ise şunu çalıştır gibi.

Ardışıl diyagramda koşullu mesaj bir kutu içine alınır. Kutunun adı "option" anlamında `opt`, koşul da yine köşeli parantezle kutunun üzerine yazılır. Koşul doğruysa metot çağrısı gerçekleşir, doğru değilse gerçekleşmez.

3.5 Karşılıklı Dışlamalı Mesajlar (if-else)

Peki, "if" varsa "else" de var mıdır? Vardır. Buna karşılıklı dışlamalı (mutually exclusive) mesajlar denir, çünkü iki dal aynı anda çalışmaz.

İşbirliği diyagramında dallar `1a` ve `1b` diye numaralanır. Hocanın örneğinde koşul doğruysa `1a` ile n2'den bir işlem çağrılır. Koşul yanlışsa `1b` ile n3'ten başka bir işlem çağrılır. Bu ikinci işlem çalışırken de n3, n4'ten bir çağrı yapar: `1b.1`.

Ardışıl diyagramda "optional" değil, "alternate" anlamındaki `alt` kutusu kullanılır. Kutu ortadan kesikli bir çizgiyle ikiye bölünür. Üst bölme koşulun doğru, alt bölme koşulun yanlış olduğu durumdur.

Hoca burada slayttaki kendi çiziminde bir hata fark etti ve derste düzeltti. Ardışıl diyagramda `1b.1` mesajı n1'den n4'e çizilmişti. Oysa `1b`'nin içinde `1b.1` olması, mesajı n3'ün yolladığını gösterir. Doğrusu, n3'ün çubuğunun içinden n4'e giden bir oktur. n1'den n4'e gidiyor olsaydı işbirliği diyagramının da ona göre çizilmesi gerekirdi. Bu küçük hata, numaralandırmanın nasıl okunacağını da gösteriyor: Numaranın başındaki parça, mesajın hangi metodun içinden yollandığını söyler.

3.6 Döngü

Elinizde bir koleksiyon, bir liste ya da bir küme varsa ve bütün elemanları için çağrı yapmak istiyorsanız döngü gerekir. Örneğin bir listedeki bütün derslerin notlarını alıp ağırlıklı not ortalaması hesaplayacaksanız, her elemanın değerini sorgulamanız gerekir.

İşbirliği diyagramında bunun için **yıldız (*)** kullanılır. Yıldız "hepsi" demektir. Hocanın örneği: Barındırdığım bütün sipariş kalemleri için "alt toplam al" metodunu çağır. Bu aslında bir foreach döngüsünün tasarımıdır.

Ardışıl diyagramda `loop` kutusu kullanılır ve “alt toplam al” çağrısı bu kutunun içine yazılır. Eski UML 1 sürümlerinde ise koleksiyon, nesne kutusunun arkasına üst üste başka kutular koyularak gösterilirdi. “Bu bir nesne listesidir, n tanedir” demek için böyle yapılırdı.

Döngüde bir koşul varsa, bu da koşullu mesajlardaki gibi köşeli parantez içinde verilir. Hocanın örneği “birden n’e kadar”dır, yani `[i = 1..n]` gibi. Ardışıl diyagramda da aynı biçimde tanımlanır.

> **Ek bilgi (derste anlatılmadı):** `opt`, `alt` ve `loop` kutuları UML 2’de büyük olasılıkla “birleşik parça” (combined fragment) adıyla geçer. Kutunun sol üst köşesindeki kısa ad işlecin türünü, köşeli parantezdeki koşul ise o bölmenin ne zaman çalışacağını söyler. Hocanın anlattığı üç kutu bu ders için yeterlidir.

3.7 Çizim Araçları

Hoca, bu diyagramları çizmek için hafif ve ücretsiz bir uygulama olan Violet UML’i önerdi. Java tabanlıdır ve diyagramlar hızlıca oluşturulur. Biraz daha gelişmişi ArgoUML’dir. Bunların dışında IBM’in ürün ailesi gibi daha profesyonel araçlar ve Eclipse eklentileri de vardır. Ama bir raporda hızlıca diyagram çizmek için Violet ya da Argo rahatlıkla yeterlidir.

4. Kullanım Senaryolarının Gerçeklenmesi

Peki hangi tasarımı, neye göre ifade edeceğiz? Yaptığımız işe **kullanım senaryolarının gerçeklenmesi** diyoruz. Analiz modelinde, iş analizinde çıkardığımız kullanım senaryolarındaki bütün sorumlulukları, yani yerine getirilmesi gereken işleri buluruz. Her biri için bir sınıf etkileşimi oluşturup diyagramlarla ifade ettiğimizde, yazılımın bütün ayrıntılı tasarımını oluşturmuş oluruz.

Kabaca söylersek tasarım, yazılım sınıflarına metotları eklemek ve senaryolardaki gereksinimleri karşılamak için bu sınıflardan türeyen nesneler arasında nasıl bir mesajlaşma olacağını, hangi sırayla kimin neyi nasıl çağıracağını belirlemektir.

4.1 İki Çeşit Sorumluluk

Bütün kavramsal sınıfları yazılım sınıfına dönüştürdüğümüzde her birine belli sorumluluklar yüklemeliyiz ki sorduğumuzda cevabını verebilsin. Genel olarak iki çeşit sorumluluk vardır: bilmemiz gerekenler ve yapmamız gerekenler.

Bilme sorumlulukları üç gruptur. Hoca her birini bir öğrenci kavramı üzerinden örnekledi:

- **Kendi özel verileri:** Öğrenci adını, soyadını, numarasını, sınıfını ve bölümünü bilmelidir, sorulduğunda söyleyebilmelidir.
- **İlgili diğer nesneler:** Öğrenci o güne kadar aldığı bütün derslerin listesini bilmelidir.
- **Hesaplanabilen, türetilebilen veriler:** Öğrenci, aldığı derslere ait nesnelerdeki notlarını bilmeli ve bunların üzerinden not ortalaması gibi bir veriyi hesaplayabilmelidir.

Yapma sorumlulukları şunlardır:

- Başka bir nesneyi yaratmak ya da yok etmek.
- Başka bir nesneyi harekete geçirmek, tetiklemek, metodunu çağırmak.
- Bir işin yapılmasını başka bir nesneye pas etmek.
- Başka nesnelerin nasıl çalıştığını denetlemek.

Karşılaştığımız davranışlar temelde bunlardır. Bunları metotlar hâlinde sarmalayıp ilgili sınıflara vermeye çalışırız. Yazılım tasarımı dediğimiz iş budur.

4.2 İteratif Çalışma

Sorumlulukları metotlarla karşılıyoruz. Kavramsal sınıfları alıp çeşitli sorumluluklar vererek ayrıntılı yazılım tasarımının içine bıraktığımızda, onlar birer yazılım sınıfı hâline gelir. Bir senaryonun çalışması, tek bir sınıfın tek bir metodunu yazmak demek değildir. O metot sorumluluğu yerine getirmek için başka nesnelerin başka metotlarıyla iş birliği yapabilir.

Bunu bir kerede yapmak zordur, bu yüzden genellikle iteratif çalışılır:

1. Elimizdeki senaryoların içinden en basitiyle başlanır.
2. Önce onun ana akışı, yani ana başarılı akış (main success flow) ele alınır ve gerçekleştirilir.
3. Bir sonraki iterasyonda alternatif akışlar ele alınır: Şu hata oluşursa, nakit ödüyorsa, kartla ödüyorsa gibi. Bunlarla ilgili sorumluluklar da atanır.

Senaryo büyükse alternatif akışlar da çok olacağı için bu birkaç iterasyon sürebilir. Basit senaryolar tek iterasyonda bitebilir.

Bütün senaryolar işlenip sorumluluklar yazılım sınıflarına atandığında, kodlanmak üzere hazırlanmış tam bir çözüm taslağı elde edilir. Tasarım budur.

Bu iş bitince tersine de gidilebilir. Tasarım sırasında unuttuğumuz, atladığımız kavramlar ortaya çıkarsa, bunlar kavram modeline geri eklenir. Yani bir geri besleme vardır ve analiz diyagramının son hâli tasarım bittikten sonra oluşturulabilir. Ama analiz modelinde davranış olmasını beklemeyiz, metotlar yazılım sınıflarının taşıması gereken özelliklerdir. Geriye yalnızca yeni kavramlar eklenir, metotlar eklenmez.

5. Satış Örneği ve GRASP'a Giriş

5.1 Senaryo

Bir sorumluluğun nasıl atandığını ve iş birliklerinin nasıl ortaya çıktığını görmek için analizde çalıştığımız satış senaryosuna dönüyoruz. Müşteri elindeki ürünlerle kasaya gelir. Kasiyer yeni bir satış başlatır ve ürün kimliğini girer. Sistem satışı bir satır olarak kaydeder. Ekranda ürünün tanımını, fiyatını ve toplam tutarı gösterir. Bu, belirli bir fiyat politikasına göre hesaplanır. Bütün ürünler için bu bir döngü hâlinde tekrarlanır.

Senaryoya bakınca yapılması gereken işler, yani sorumluluklar görülür: bir başlatma sorumluluğu, bir ürünün okutulması sorumluluğu. Hoca özellikle senaryonun sonundaki "System presents total with taxes calculated" adımına odaklandı. Burada toplam fiyatın (total price) hesaplanması ve gösterilmesi gerekir. Yanı sıra belki bir vergi hesabı da gerekir. Vergi için bir dış sisteme, bir muhasebe sistemine ya da bir vergi merkezine sormamız gerekebilir. Hocanın örneğinde sistem "bu bir bardaktır, vergisi kaçtır?" diye sorduğunda "%8" cevabını, "bu bir telefondur" diye sorduğunda "%18 KDV, şu kadar ÖTV" gibi bir cevap alır. Bu da başka bir sorumluluktur. Hoca vergiyi bu aşamada kenara bırakıp yalnızca **satışın toplam bedelinin hesaplanmasına** baktı.

Bu sorumluluğu kim taşımalı? Analizden gelen kavramsal sınıflarımız kasa, dükkan, satış, satış kalemi, ürün ve ürün tanımlarından oluşan katalogdur. Bu adaylardan hangisinin toplam tutarı hesaplaması gerektiğini ve bunu yaparken başka hangi kavramlarla etkileşime geçeceğini düşünüp sonucu bir tasarım çözümü olarak yazmamız gerekir. Bunu da genel prensipler ve kalıplar çerçevesinde yaparız.

5.2 GoF ve GRASP

Tasarım kalıplarının temeli **Gang of Four (GoF)** kalıplarıdır. Sonradan daha da genişlemiştir. Toplam 23 GoF kalıbı vardır. Her tür yazılımı tasarlarken sık karşılaşılan 23 genel problem ve bunlar için en iyi çalıştığı bilinen 23 çözüm olarak düşünebilirsiniz. Bir iş yapılacağı zaman "şu kalıbı uygulayalım, şu kalıpla şu kalıbı birlikte uygulayalım" diye konuşarak tasarımı çok daha hızlı oluşturabilirsiniz. Ayrıntıları Yazılım Mühendisliği dersinde göreceksiniz.

> **Ek bilgi (derste anlatılmadı):** "Gang of Four" adı büyük olasılıkla, bu 23 kalıbı 1994 tarihli *Design Patterns* kitabında yayımlayan dört yazara (Gamma, Helm, Johnson, Vlissides) gönderme yapar. Hoca adın nereden geldiğine girmedi.

Hoca bu derste 23 GoF kalıbı yerine 9 genel sorumluluk atama prensibini anlattı. **GRASP**, *General Responsibility Assignment Software Patterns* kısaltmasıdır. Türkçeye "genel sorumluluk atama yazılım kalıpları" diye çevrilebilir. Bunlar yeni ya da özel şeyler değildir, çoğuna nesneye yönelik programlamadan aşinasınızdır. Hangi probleme çözüm getirdiğini anlatan kolay isimleri vardır. Her kalıp bir soruyla başlar ve bu soruya cevap olarak bir çözüm önerir.

No	Türkçe adı	Asıl adı
1	Uzman	Expert
2	Yaratıcı	Creator
3	Az Bağımlılık	Low Coupling
4	İyi Uyum	High Cohesion
5	Denetçi	Controller
6	Çok Şekillilik	Polymorphism
7	Yapay Sınıf	Pure Fabrication
8	Dolaylılık, Ara Birim	Indirection
9	Değişimlerden Korunma	Protected Variations

6. Uzman (Expert)

Soru: Bir nesneye sorumluluk atamanın temel prensibi nedir? Örneğimizde satışın toplam bedelini kim hesaplayacak, yani o mesaj kime gitsin?

Çözüm: Sorumluluğu bilginin uzmanına, yani onu yerine getirecek veriye sahip olan sınıfa atayın.

Hoca bunun yeni bir şey olmadığını, en mantıklı olanı söylediğini belirtti. Uygulama sırası da şöyledir: Önce yazılım sınıflarımıza bakarız. Bunu karşılayan bir yazılım sınıfı henüz yoksa analiz modeline döner, gerekli bilginin hangi kavramda durduğuna ya da hangi kavramın onu okuyup oluşturabildiğine bakarız. En uygun kavramı alır, davranışı ona ekleriz. Böylece o kavramsal sınıf artık bir yazılım sınıfı olur ve tasarım modelinde yer alır.

Bunun kazancı, gerçek dünyadaki kavramlarla yazılım uzayındaki sınıflar arasındaki **temsil farkının** mümkün olduğunca azalmasıdır. Yani gerçek dünyanın yazılımdaki modelini birebir kurmaya çalışırız.

6.1 Sorumlulukların Zincirleme Atanması

`getTotal` kimde olmalı? Satışın toplam bedelini belirlemek için `getTotal` diye bir metot gerekir. Satış kalemine versek o yalnızca tek satırı bilir. Ürün tanımına versek o yalnızca bir ürünün tanımını bilir. Bütün kalemlere ve onlar üzerinden bütün ürün tanımlarına erişebilen satıştır. Metot `Sale` sınıfına verilir ve sınıf diyagramında üçüncü bölmesine yazılır.

`getSubtotal` kimde olmalı? `Sale` işi nasıl yapacak? İçinde tuttuğu satırlar, ürün okuttukça oluşmuştu. Bu listedeki her elemana "kardeş, senin tutarın ne kadar?" diye sorması gerekir. Hocanın anlatımıyla kasiyere ürünleri getirdim: 3 kalem, 5 silgi ve 2 kalemtıraş. Hoca toplamı önce harflerle `3K + 5T + 2KT` diye ifade etti: 3 tane kalem, 5 tane silgi ve 2 tane kalemtıraşın tutarları toplanır. Her satır kendi tutarını bildiği için bu işin uzmanı satırdır. `SalesLineItem` sınıfının üçüncü bölmesine `getSubtotal` eklenir.

`getPrice` kimde olmalı? Satır kendi alt toplamını nasıl hesaplar? Miktar kendisindedir, örneğin "kalem, miktar 3". Ama bir tanesinin kaç para olduğunu bilmez. O bilginin uzmanı ürün tanımıdır (`ProductDescription`). Her satır bir ürün tanımına bağlıdır. Bu yüzden `getPrice` ürün tanımına eklenir.

Sonuçta tek bir sorumluluk için üç sınıfa üç metot atanmış olur. Herkes uzmanı olduğu konuda konuşur.

6.2 Çalışma Sırası ve Sayısal Örnek

Hoca senaryoyu sayılarla adım adım yürüttü. `t = sale.getTotal()` çağrıldığında `Sale` bir değişken tutar, listedeki satırları for-each ile dolaşır ve alt toplamları bu değişkende biriktirir.

Verilen: 3 kalem (birim fiyat 10 TL), 5 silgi (birim fiyat 5 TL), 2 kalemtıraş (birim fiyat 20 TL).

İşlem:

1. `Sale`, birinci satıra `getSubtotal` der. Satır, ürün tanımına `getPrice` ile sorar ve 10 cevabını alır. Miktar kendisinde olduğu için $3 \times 10 = 30$ hesaplar ve "ben 30 lirayım" diye döndürür.
2. `Sale`, ikinci satıra sorar. Silgi fiyatı 5 liradır. $5 \times 5 = 25$ döner.

3. `Sale`, üçüncü satıra sorar. Kalemtıraş fiyatı 20 liradır. $2 \times 20 = 40$ döner.
4. `Sale` alt toplamları toplar: $30 + 25 = 55$, $55 + 40 = 95$.

Sonuç: `getTotal`, 95 değerini kendisini çağırana döndürür, yani $t = 95$ TL olur.

Satır	Miktar	Birim fiyat (TL)	Alt toplam (TL)
Kalem	3	10	30
Silgi	5	5	25
Kalemtıraş	2	20	40
Toplam			**95**

Hesap Python ile doğrulandı. Aşağıdaki kod derste yazılmadı. Hocanın "bir değişken tutacağım, for-each ile alt toplamları toplayacağım" anlatımından kuruldu, derlendi ve çalıştırıldığında 95 sonucunu verdi. Tutar tipi burada basitlik için `int` olarak alındı.

```
class SalesLineItem {
    private final ProductDescription description;
    private final int quantity;

    SalesLineItem(ProductDescription description, int quantity) {
        this.description = description;
        this.quantity = quantity;
    }

    int getSubtotal() {
        return quantity * description.getPrice();
    }
}

class Sale {
    private final List<SalesLineItem> lineItems = new ArrayList<>();

    int getTotal() {
        int total = 0;
        for (SalesLineItem item : lineItems) {
            total += item.getSubtotal();
        }
        return total;
    }
}
```

Hocanın özetiyle, bu parçacık yazılım tasarımında tek bir sorumluluğu yapan parçacıktır. Tasarımcı aslında programcıya şunu söylemiş olur: Bu metodu yazarken listede for-each ile `getSubtotal` yap, her `getSubtotal` içinde de ürün tanımından `getPrice` al.

7. Yaratıcı (Creator)

Soru: Bir sınıftan nesne yaratma sorumluluğu kimdedir? Şimdiye kadar nesneler uygulamada zaten varmış gibi düşündük. Ama yeni nesneler de yaratılması gerekir. Satış nesnesini kim yaratmalı? Satış kalemlerini kim yaratmalı?

Çözüm: Bir B sınıfına, A nesnelerini yaratma sorumluluğunu şu durumlarda atayın. Yani `new A()` yazdığınız yer B olsun:

- B, A nesnelerini içeriyorsa.
- B, A'ların kaydını tutuyorsa.
- B, A'ları kullanıyorsa.
- A'nın yaratılması için gereken başlangıç verilerine B sahipse.

> **Ders sonrası not:** Hoca bu maddeleri "ve" ile bağlayarak okudu. Maddelerin hepsinin birlikte sağlanması büyük olasılıkla gerekmez, biri yeterli sayılır. Hocanın kendi örneğinde de gerekçe olarak yalnız `Sale`'in satırları barındırması gösterildi.

Örnek: Çok sayıda satış kalemini kim barındırıyor? Satış. O hâlde onları da satış yaratmalıdır. Kasaya ürünü okuttuğumuzu düşünelim. Kasiyer "3 ×" yazar, barkodu girer ve enter'a basar. Bu sırada kasada bir metot çalışır ve bu metot satış nesnesine `makeLineItem` çağrısı yapar: Böyle bir ürün geldi, şu kadar tanesiyle bir satır oluştur. Satış kendi içinde bir liste tutar ve bu listede `SalesLineItem` türünde elemanlar vardır. Bunları barındırdığı için yaratan da odur. "new" işlemi diyagramda bir `create` çağrısıyla gösterilir.

Burada ardışıl diyagramla ilgili önemli bir ayrıntı vardır. Yeni nesnenin yaşam çizgisi diyagramın en üstünde değil, `create`'in çağrıldığı yükseklikte başlar. Çünkü o nesne daha önce yoktu, ancak bu metot çalışınca yaratıldı.

8. Az Bağımlılık (Low Coupling)

Üçüncü ve dördüncü prensipler birbirine çok sıkı bağlıdır ve doğrudan nesneye dayalı prensiplerdir.

Soru: Diğer sınıfların değişimlerinden etkilenmemek ve mümkün olduğunca tekrar kullanılabilir olmak nasıl sağlanır? Bir kavram diğer kavramlara nasıl az bağımlı tutulur?

Çözüm: Sorumlulukları bağımlılığı en aza indirecek şekilde atayın. Uzmanları belirlerken de tekil, atomik birimler oluşturmaya özen gösterin.

Bağımlılık (dependency), bir sınıfın kendi iç işleri için başka sınıfları ne kadar kullandığı ve onlar hakkında ne kadar bilgi içermesi gerektiğidir. Ne kadar çok bilirse o kadar kötüdür, çünkü:

- Bir sınıftaki değişim, ona bağlı sınıflarda da çok fazla değişime yol açar.
- Sınıfları ayrı ayrı anlamlandırmak zorlaşır.
- "Böyle bir kavram bulmuştum, başka uygulamada da kullanayım" dediğinizde yeniden kullanım zorlaşır.

A'nın B'ye bağımlılığı şu durumlarda oluşur:

- A'nın içinde B tipinde bir üye, yani özellik varsa.
- A, B'nin bir metodunu çağırıyorsa.
- A'nın bir metodunda B tipinde parametre varsa.
- A'nın bir metodunda B tipinde yerel değişken varsa. Bu bağımlılık yalnız metot ömrü boyunca sürer.
- A doğrudan ya da dolaylı olarak B'den türetildiyse, yani bir kalıtım ilişkisi varsa. Bu durumda A, soyuna bağımlıdır.

Hoca kısa süreli bağımlılığı özellikle ayırdı: Metot süresince oluşan bağımlılık kalıcı olandan daha azdır. "Ne kadar az, o kadar iyi."

8.1 Ödeme Örneği

Satırlar okutuldu, toplam tutar ve vergiler hesaplandı, sıra ödemede. Ödeme gerçek hayatta kasaya yapılır. O hâlde `makePayment` sorumluluğu kasaya (`Register`) gelir.

İlk tasarım: `makePayment` çağrısı `Register`'a girer. Elinde bilgi olduğu için `Register` bir `Payment` nesnesi yaratır. Sonra `sale.addPayment(p)` çağırarak bu nesneyi satışa bağlar, yani satışın bir üyesi hâline getirir.

Bu tasarım gerçek hayatı iyi yansıtır ama bir sorunu vardır. `Register`, ödemeyi kendisi yarattığı için `Payment`'a bağımlı olur. Kasa nesnesi ödeme nesnesiyle ilgili her şeyi görür hâle gelir. Gerek var mı? Uygulamayı tasarlarken bunu tartışmak gerekir. `Register`, ödemeyle ilgili hiçbir şey yapmıyorsa bağlı olmasına da gerek yoktur.

> **Ders sonrası not:** Hoca bu tasarımın "iyi uyumu biraz bozduğunu" söylerken gerekçe olarak bağımlılığı gösterdi. Burada doğrudan söz konusu olan az bağımlılık

prensibidir. Uyum yönü aşağıda 9.1'de ayrıca ele alındı. Yine hoca "`Register` değişirse `Payment` da etkilenecek, `Payment` değişirse `Register` da etkilenecek" dedi. Bu tasarımda bağımlı olan taraf `Register`'dır. Etkilenme yönü de hocanın az sonra söylediği gibidir: "Oradaki değişim beni etkileyecek." Yani `Payment`'taki değişim `Register`'ı etkiler.

Daha iyi tasarım: İşi gerçek sahibine, gerçek uzmanına pas ederiz. Yapma sorumluluklarında saydığımız "işi başkasına ötelemek" tam olarak budur. `makePayment` kasaya gelir, kasa hiçbir şey yapmadan sahibi olduğu satıştaki `makePayment` metodunu çağırır. Ödemeyle sıkı ilişkide olan ve onu en çok kullanacak olan satıştır. `Payment`'ı `Sale` yaratır.

```
class Register {
    private final Sale sale;

    Register(Sale sale) {
        this.sale = sale;
    }

    void makePayment(int amount) {
        sale.makePayment(amount);
    }
}
```

Bu parça da hocanın anlatımından kuruldu ve `Sale` ile birlikte derlendi. `Register` artık `Payment` hakkında hiçbir şey bilmez, bilmek zorunda da değildir. Bilseydi ona bağımlı olurdu ve oradaki bir değişim onu etkilerdi. `create` işini kendisi yapmaz, onu gerçekten en çok kullanacak olana delege eder.

9. İyi Uyum (High Cohesion)

Soru: Yazılım tasarımındaki karmaşıklık nasıl idare edilebilir?

Çözüm: Sınıf içinde iyi uyum olacak şekilde sorumlulukları atayın.

Bir kavramdaki sorumluluklar birbiriyle ilgili olmalı ve belli bir konu başlığına yoğunlaşmalıdır. Bir sınıf birbiriyle ilişkisi olmayan bambaşka işler yapıyorsa ya da beklenenden çok fazla sorumluluk alıyorsa, o sınıfın uyumu kötüdür.

Bunun sonu karmaşık ve büyük sınıflardır. En ucu **god class**, yani her şeyin tek sınıfta toplanmasıdır. Bu istenen bir tasarım değildir, çünkü iyi uyuma sahip olmayan bir sınıf:

- Zor anlaşılır.
- Bakımı zordur.
- Taşıdığı alakasız sorumluluklar yüzünden başka yerde zor kullanılır.
- Değişimlere karşı savunmasızdır. İçinde çok iş olduğu için başka yerlere de bağımlıdır.

O hâlde sınıfları, belirli bir grup sorumluluğu yerine getiren yapılar olarak modellemeye çalışırız.

9.1 İyi Uyumdan Vazgeçilen Durumlar

Bazı durumlarda iyi uyumdan feragat edilebilir. Hoca iki örnek verdi:

- **Veri tabanı işleri:** Satış, ödeme, kasa gibi gerçek kavramlarınız var. Satışla, ürünle ilgili şeylerin veri tabanına yazılması ve okunması gerekiyor. Her kavramın kendi içinde veri tabanına erişmeyi bilmesi gerekir. "Ben satış nesnesiyim, niye veri tabanıyla uğraşayım?" diye düşünerek bütün veri tabanı işlerini gerçekte olmayan bir yapay sınıfa verebilirsiniz. Böylece belirli bir grup iş, kavram bazlı değil iş bazlı modellenmiş olur. Bütün veri tabanı işleri de belli bir ekibe ya da belli bir programcıya verilebilir.
- **Dağıtık sistemler:** İyi uyum yüzünden çok fazla nesne çok fazla etkileşime geçebilir. Ağ trafiğini, yani mesaj trafiğini azaltmak için uyumsuz da olsa daha fazla iş yapan, daha büyük nesneler tercih edilebilir.

Yine de mümkün olduğunca sınıfları kendi içinde ilgili işler yapacak biçimde oluşturmakta fayda vardır.

> **Ders sonrası not:** Hoca yapay veri tabanı sınıfı için burada "uyumu biraz bozuyor, çünkü bütün sınıflar ona bağımlı oluyor" dedi. Yapay Sınıf bölümünde ise aynı çözüm için "uyumu artırır" dedi. Gösterilen gerekçe, yani herkesin yapay sınıfa bağımlı olması, uyumla değil bağımlılıkla ilgilidir. İkinci ifade, veri tabanı işlerinden kurtulan kavram sınıflarının kendi uyumu için doğrudur.

9.2 Aynı Örnek, İşbirliği Diyagramıyla

Hoca ödeme örneğini bu kez işbirliği diyagramıyla ve uyum açısından yeniden ele aldı. Eski tasarımda kasa, bir `Payment` yaratıp satışa "al kardeşim" diye veriyordu. Oysa kasanın kendi iç uyumu açısından bu alakasız bir iştir: "Ben bunu niye yarattım? Böyle bir sorumluluğum niye olsun?" Yeni tasarımda kasanın metodunun içinde yalnızca `sale.makePayment` çağrısı vardır. Yaratma ve ödemeye sahip olma sorumluluğu satışa geçer. Kasa, kendi içinde tutarlı metotlarla yoluna devam eder. Satış da sıkı biçimde bağlı olduğu ödemeyi kendisi yaratır.

> **Ders sonrası not:** Hoca burada "kasanın satışa az bağlı olması" dedi. Örneğin gösterdiği şey, kasanın **ödemeye** az bağlı olmasıdır. Kasa satışa bağlı kalmaya devam eder, çünkü `sale.makePayment` çağrısını yapar. Hoca 8.1'deki anlatımında da "kasanın ödemeye az bağımlı kalması için" demişti.

Hocanın benzetmesi gerçek hayattan: Bir iş yapılacağı zaman herkes o işten kaçar ve "al sen yap, al sen yap" diye başkasına atar. Burada da benzer bir şey vardır: "Ben niye bununla uğraşayım? Bu sorumluluk niye bende olsun? Zaten bende o bilgi yok ki, al sen yap."

> **Hocanın vurgusu:** 1, 2, 3 ve 4 numaralı GRASP prensipleri bir arada düşünülerek sorumluluk ataması yapılmalıdır.

10. Denetçi (Controller)

Örneklerde hep "kasiyer düğmeye bastı", "satışı tamamla butonuna bastım" dedik. Yani ilk mesajı gönderen bir olay vardır.

Soru: Bu olayları kim karşılasın? Bu işin sorumlusu, uzmanı kimdir?

Çözüm: Sistem olaylarını algılayıp değerlendirme ve ilgili sınıftan ilgili metodu çağırma sorumluluğunu şu iki seçenekten biriyle oluşturun:

- **Ön yüz denetçisi (façade controller):** Bütün sistemi ya da alt sistemi temsil eden bir denetçi sınıfı oluşturun.
- **Senaryo ya da oturum denetçisi (use case controller, session controller):** Sistem büyükse bütün denetleme işlerini tek yere koymak, denetçinin farklı alt parçalara çok bağımlı olmasına yol açar. Bunu engellemek için her senaryoya ayrı bir denetçi oluşturun. Bu durumda sistemde birden fazla denetçi olur.

Sistem olayları, aktörler tarafından üretilen ve sistemde etkileşimi başlatan olaylardır. Bir butona tıklamak ya da bir şeyi sürüklemek gibi. Yaptığınız her eylem aslında bir sistem olayı yaratır.

Kaç denetçi olacağına şöyle karar verilir: Çok fazla olayınız yoksa tek denetçi kullanabilirsiniz. Denetçinin uyumu bozulacak kadar çok olayınız varsa birden fazla denetçi daha uygundur. Senaryolarda birbirinden farklı sistem kontrolleri varsa her senaryoya ayrı bir denetçi yaratabilirsiniz.

Dikkat edilecek nokta şudur: İşi denetçi yapmaz, yalnızca aktarır. Ödeme örneğindeki kasa gibi, ilgili mesajı işten sorumlu nesneye iletir.

10.1 Denetçi Kullanmanın Faydaları

- **Soyutlama:** Arayüz katmanı ile uygulama katmanı birbirinden ayrılır. İş mantığı ile sunum mantığı birbirinden bağımsız geliştirilip değiştirilebilir.
- **Esneklik:** Denetçi görevini doğrudan bir arayüz nesnesine, mesela bir pencere (window) nesnesine de verebilirsiniz. Ama o zaman yeniden kullanılabilirliği ve esnekliği kaybedersiniz, çünkü iş o pencereye bağlanır.

- **Mantık sırası kontrolü:** İş sırasının düzgün işletilmesi denetçide garanti altına alınabilir. Örneğin sipariş vermeye basmadan ödemeye basılmamalıdır. Böyle mantık hataları denetçide kontrol edilebilir.

10.2 Satış Sisteminde Denetçi

Satış sisteminde satışın tamamlanması, ürün girilmesi, yeni satış başlatılması, ödeme, iade gibi sorumluluklar vardır. Bunları kim karşılayacak? Hepsini `Register`'a verebilirim. Bu yetmiyorsa, iyi uyumu bozuyorsa ve daha çok işim varsa alt parçalara bölerim: Satış işlemleri için bir denetçi, iade işlemleri için başka bir denetçi gibi. Bunlar senaryo denetçileridir.

Denetçinin yeri şöyledir. Üstte kullanıcı arayüzü katmanı vardır. Kasiyerin ekranında barkod ve miktar alanları bulunur. Gerçek hayatta "3 ×" yazıp barkodu okutur, burada basit olsun diye miktar alanına 3 yazıp **Enter Item** butonuna basar. Bu tıklama bir sistem olayıdır ve bir `JFrame`'de oluşur, `actionPerformed` metoduna gelir. Birinin bunu karşılaması gerekir. Bu olayı karşılayıp kontrol eden nesneye denetçi diyoruz. Hocanın örneğinde bu nesne `Register`'dır.

Üç katman oluşur:

1. **Ön yüz (user interface) katmanı**
2. **Denetçi (controller) katmanı**
3. **İş mantığı (business logic) katmanı**

Denetçi, arayüz ile iş mantığı arasında bir **tampon bölgedir**. Yarın bir mobil arayüz yaparsanız o da yalnızca denetçiye erişir. Denetçi gerekirse kendi içinde yeni bir metot ekler ya da bir metodunu değiştirir, ama iş katmanındaki nesneler bundan etkilenmez.

Hoca bir örnek daha verdi: Migros'lardaki self-servis kasalar gibi bir kiosk koyuldu diyelim. Ekranı, kart sokma yeri, fiş basma yeri, poşet koyma alanı var. Ürün okutulduğunda yeni olaylar oluşur. Bunları kim karşılayacak? İş mantığında hiçbir şey yapılmaz, olaylar yine denetçiye gider. Hatta o ön yüzü bambaşka bir şirket geliştiriyor olabilir.

Hoca bunların hazır bir reçete olmadığını da söyledi. "Otururuz, tam olarak böyle yaparız" denemez, bunlar yol yordamdır. Her probleme göre en uygun çözüm geliştirilir. Amaç değişimlerden korunacak ve kavramları birbirinden soyutlayacak katmanlı bir yapı kurmaktır.

11. Çok Şekillilik (Polymorphism)

Soru: Tiplere bağlı alternatifler nasıl ele alınmalı? Sisteme kolay monte edilen, duruma göre değişebilecek parçalar nasıl yaratılıp eklenmeli?

Çözüm: Alternatif davranışlar sınıflara ve koşullara bağlı olarak değişiyorsa, bunları polimorfik metotlarla gerçekleyin. Yani if-else kullanmayın. Tek bir davranış olsaydı çok şekilliliğe gerek kalmazdı. Çok şekillilik, kolay monte edilebilir parçalar oluşturur.

Vergi örneği: Vergi hesabını bir sistemle, başka bir sistemle ya da ileride gelecek yeni sistemlerle yapabilirim. Bunları polimorfik geliştirir ve uygulamada hepsini ortak bir arayüzden kullanırım. `Sale`, bu arayüzle bağlantılıdır. Onun için önemli olan sorumluluğun yerine getirilmesidir: `getTaxes` çağırır ve bir vergi listesi gelir. Bu sonucun hangi adaptörün arkasındaki hangi sistemle konuşularak üretildiğini bilmek zorunda değildir.

Kredi kartı örneği: Bir kredi kartı ödeme adaptörü düşünün. Arkasında Visa var, MasterCard var. Sonra biri Troy çıkardı, o geldi. Yarın Diners Club, American Express gelebilir. Bir süre sonra "biz bununla çalışmıyoruz" da denebilir. Sizin için önemli olan kartla ödeme almaktır. Visa'nın doğrulamayı ya da provizyonu nasıl yaptığı sizi ilgilendirmez. Bunlardan soyutlanmak istersiniz, çok şekilli metotlar da bunu sağlar.

Hoca gerçek çözümlerin bu kadar basit olmadığını, ancak birkaç GoF kalıbı birlikte kullanılınca anlam kazandığını ekledi. Bu aşamada bu kadarını bilmek yeterlidir.

12. Yapay Sınıf (Pure Fabrication)

Soru: Uzman kalıbının önerdiği çözüm, Az Bağımlılık ve İyi Uyum kalıplarının önerileriyle çelişebilir. Bu durumda sorumlulukları hangi sınıfa atayalım?

Çözüm: Gerçekte olmayan, yapay bir sınıf oluşturun.

Yine veri tabanı örneği: Her kavram veri tabanına ayrı ayrı bağlanırsa her biri veri tabanına bağımlı olur. Bunun yerine bağlantı açma, tablodan okuma, tabloya yazma, insert, update, delete gibi işlerin hepsini gerçekte olmayan bir sınıfa veririm. Veri tabanıyla o konuşur. Bütün uygulama katmanı da yalnızca bu aracıyla konuşur: "Al şunları yazdır, ben bunları okumak istiyorum." Bu, sınıfların kendi içindeki uyumu artırır. Ayrıca her sınıfın veri tabanı işlemlerini içermesi gerekmediği için karmaşıklığı ve bağımlılığı düşürür.

Ama bunu fazla yaparsanız gerçek dünyadaki kavramsal modellemeden uzaklaşırsınız: Veri tabanı işlemleri sınıfı, ödeme işlemleri sınıfı, vergi işlemleri sınıfı, raporlama işlemleri sınıfı, ürün okuma işlemleri sınıfı… Bu yol sizi kavramsal çözümlemeden işlevsel çözümlemeye götürür. Eski alışkanlıklarla bu tercih edilebilir, ama sonuçları kötüdür: Gerçek dünya çözümünün yazılıma taşınmasından uzaklaşırsınız, anlaşılırlık azalır ve yazılım sınıflarının birbirine bağımlılığı yükselir.

> **Hocanın vurgusu:** Yapay sınıfları gereğinden fazla kullanmayın. Yalnız gerektiği yerde, gerektiği kadar, bir ara katman ya da adaptör mantığıyla oluşturun. Sürekli yapay sınıf yapıp işlevsel çözümlemeye dönmemeye gayret edin.

13. Dolaylılık ve Ara Birim (Indirection)

Soru: Kolay değişebilen iki birim varsa aralarındaki bağımlılığı nasıl azaltabilirsiniz?

Çözüm: Sorumlulukları bir ara birim nesnesine atayın.

Hocanın sürekli söylediği "araya değişimlerden koruyacak bir adaptör koyun" önerisinin resmî tanımı budur. Ara birim aslında yapay bir sınıftır, gerçekte yoktur. Ama yazılımda değişimlerden korunmayı, iyi uyumu ve az bağımlılığı sağlar. Bu yüzden yerinde ve dozunda kullanılır. Yapay veri tabanı sınıfı, bütün iş katmanı ile veri tabanı arasında bir ara birimdir. Vergi adaptörleri ve kredi kartı adaptörleri de bu prensiple ortaya çıkar.

> **Ders sonrası not:** Hoca "controller yapay bir sınıf, gerçekte yok" dedi. Bu her denetçi için geçerli değildir. Dersin kendi örneğindeki denetçi `Register`, analizden gelen kasa kavramıdır. Ön yüz ya da senaryo denetçisi olarak ayrıca yazılan sınıflar için ifade doğrudur.

GoF kalıplarından Adapter, Façade ve Observer gibi kalıplar, 8 numaralı prensibin özelleşmiş hâlleridir. Bu prensip, birkaç GoF kalıbının süzülmüş bir anlatımıdır.

Örnek: Ben bir satış nesnesiyim ve toplam tutarı hesaplama sorumluluğum var. Satırlardan fiyatları topladım, sıra vergide. Bir adaptörün `getTaxes` metodunu çağırırım. Hangi adaptör olduğunu bilmek zorunda değilim, bunlar çok şekilli olacak. Bu çözümde `TaxMasterAdapter` sınıfından türeyen bir nesneyle konuşuyorum. O nesne TCP soketiyle bambaşka bir sistemle konuşur ya da bir web servisini çağırır, benim için önemli değildir. Bana gelecek vergi listesine bakarım. Yarın dış sistem değişirse yalnız adaptörü değiştiririm. İş mantığı hiç değişmeden devam eder.

14. Değişimlerden Korunma (Protected Variations)

Soru: Nesneler, değişimler ve kararsızlıklar birbirini en az etkileyecek şekilde nasıl tasarlanır?

Çözüm: Değişime açık, kararsız noktaları belirleyin. Sorumlulukları bu noktaların etrafında kararlı arayüzler oluşacak şekilde atayın.

Hoca bunu, ders boyunca anlatmaya çalıştığı yaklaşımın tek cümlelik özeti olarak sundu. Çok şekilli vergi adaptörleri ve kredi kartı ödeme adaptörleri bunun örnekleridir. İleride değişebilecek, sayıları artıp azalabilecek ya da çalışma şekilleri değişebilecek yerler kararsız noktalardır. Bunların etrafına sizi koruyacak arayüzler, adaptörler ve tampon bölgeler koyarsınız.

> **Hocanın vurgusu:** Bunu da abartmayın. Katman üstüne katman yapmayın, bir adaptörle, bir katmanla çözün.

Kapsülleme (encapsulation), veri gizleme (data hiding) ve çok şekillilik gibi pek çok prensip ve kalıp da bu prensibe dayanır ya da ondan beslenir.

15. Yabancılarla Konuşma Prensibi

Hoca, GRASP'ın dokuz maddesinin dışında, yeri gelmişken bağımlılığı azaltan bir tasarım kuralından daha bahsetti: Bir nesne yalnızca tanıdığı nesnelere mesaj göndermeye özen göstermelidir.

Bir nesnenin tanıdıkları şunlardır:

- Kendisi.
- Metodunun bir parametresi.
- İçinde barındırdığı bir üye, yani bir özelliği (attribute).
- İçinde barındırdığı bir listenin elemanı.
- Kendi yarattığı bir nesne.

Bunların dışındakiler yabancıdır ve yabancılarla konuşulmaz.

Kötü örnek: Kodlarken çok yapılan bir çağrı:

```
Money amount = sale.getPayment().getTenderedAmount();
```

Burada tanıdığım `Sale` nesnesini çağırıyorum. Ama `getPayment()` bana `Payment` türünden bir sonuç getiriyor ve bir anda hiç tanımadığım bir sınıfın nesnesinin metodunu çağırıyorum. Böylece `Payment`'a bağımlı hâle geliyorum. Yarın `getTenderedAmount` değişirse benim kodumun da değişmesi yüksek olasılıktır.

Doğrusu: Tanıdığım nesneye yeni bir sorumluluk veririm: `getTenderedAmountOfPayment`. Yalnız satışla konuşurum, "sayın satış, bana bu miktarı verir misin?" derim ve yalnız satışa bağımlı kalırım. `Payment` değişirse satış zaten ona bağımlıdır, gereğini o yapar. Ben bu değişimden korunmuş olurum. Bir yabancıyla konuşmam gerçekten gerekiyorsa, bu konuşma işi tanıdığım nesneye yeni bir sorumluluk olarak atanır.

> **Ders sonrası not:** Hoca bu miktardan "kalan miktar" diye söz etti. Metot adındaki "tendered" sözcüğü büyük olasılıkla "kalan" değil, ödeme için verilen tutar anlamındadır.
>
> **Ek bilgi (derste anlatılmadı):** Bu kural büyük olasılıkla yazılım literatüründe "Demeter Yasası" (Law of Demeter) adıyla da anılır. Yan yana zincirlenen `a.b().c()` biçimindeki çağrılar, kuralın ihlal edildiğine dair tipik bir işarettir.

16. İlklendirme Senaryosu

Dokuz prensibe göre bütün kullanım senaryolarını, yazdıysak bütün sözleşmeleri de gözden geçirir ve yapılması gereken her sorumluluğun etkileşim diyagramını çizeriz. Sonuçta bütün metotlar oluşur, ilgili sınıflara atanır, sınıflar birbirine bağlanır ve yazılım sınıf diyagramı ortaya çıkar. Etkileşim diyagramında iki sınıf arasında bir iş yapılıyorsa, tasarım sınıf diyagramında bu iki sınıf arasında bir bağlantı vardır.

Bir de sistemin başlangıç işlemleri düşünülmelidir. Önce kim yaratılacak? `public static void main` nerede duracak? Bu nesne neleri yaratacak, hangi listeleri nasıl oluşturup dolduracak, nesneleri birbirine nasıl görünür kılıp bağlayacak? Bunlar ayrı bir senaryo grubu, yeni bir kullanım senaryosu olarak yazılabilir. Genellikle tasarımın en son aşamasında belirlemek daha doğrudur.

`main`'i yazacağımız bir başlangıç nesnesi belirlenir. Program çalışınca bu nesne yaratılır. Diğer nesneleri yaratma ve bağlama sorumluluğu da ona verilir. Bu nesne programın çalışmasını denetleyen temel nesne gibi davranır. Bu görevi bir arayüz nesnesine de verebilirsiniz. Ama her durumda arayüz nesnelerine hangi denetçiyle konuşacaklarını başta söylemeniz gerekir: "Ey pencere, bak denetçi burada, bununla konuş."

Örnek (dükkan yaratılınca):

- **1:** `Store`, `ProductCatalog`'u yaratır.
  - **1.1:** Katalog, boş bir ürün tanımı listesi yaratır.
  - **1.2:** Katalog kendi `loadProductSpecs` metodunu çağırır. Bir döngü içinde bütün ürün tanımları tek tek yüklenir.
    - **1.2.1:** Her `ProductSpecification` yaratılır.

    - **1.2.2:** Yaratılan tanım listeye eklenir.
- **2:** `Store`, `Register`'ı yaratır ve `ProductCatalog` nesnesini ona parametre olarak verir: "Bak, ürün kataloğu bu."

  Ders sonrası not: Hoca kataloğun kasaya verilme nedenini, satışın satırlara ve satırların ürün tanımına soru sormasıyla açıkladı. Ama satır, ürün tanımına katalog üzerinden erişmez, ona `makeLineItem` ile verilen tanımı kullanır. Dersin bir sonraki örneğinde görüldüğü gibi kataloğu kullanan kasadır: Kasa, girilen ürün kimliğiyle katalogdan ürün tanımını alır. Kasanın kataloğu görmesinin doğrudan nedeni budur.

17. Görünürlük (Visibility)

Bir nesneye mesaj gönderebilmek için onun görünür olması gerekir. Bir B nesnesinin A nesnesine görünür olması, A'nın B'ye erişebilmesi demektir.

Görünürlük çeşitleri şunlardır:

Çeşit	B nesnesi A için nedir?	Ne kadar sürer?
Özellik görünürlüğü (attribute)	A'nın bir üyesidir	A, üyesini yaşattığı sürece
Parametre görünürlüğü	A'nın bir metodunun parametresidir	Yalnız metodun ömrü boyunca
Yerel görünürlük	A'nın bir metodunda yerel değişkendir	Metodun içinde
Global görünürlük	Global uzayda bir üyedir	Hoca süre belirtmedi

Parametre görünürlüğü, bağımlılık daha az olduğu için tercih edilir.

> **Ders sonrası not:** Hoca "genel olarak iki çeşit" diye başladı, ardından yerel ve global görünürlüğü de ekleyerek dört çeşit saydı. Listede doğru sayı dörttür.

17.1 `enterItem` Örneği

Kasaya bir ürün kimliği ve miktar girilir: `enterItem(id, quantity)`. Kasa, barkod numarasına karşılık gelen ürünün adını ve fiyatını ekranda göstermek ister. Kataloğa sorar ve sonucu kendi içinde bir değişkene atar:

```
spec = catalog.getSpecification(id);
sale.makeLineItem(spec, quantity);
```

Burada üç farklı görünürlük vardır:

- `id` ve `quantity`, `enterItem`'ın parametreleridir. Parametre görünürlüğü vardır.
- `spec`, metodun içinde yerel bir değişkendir. Kasa için yerel görünürlük vardır.
- `makeLineItem`, `spec` ve `quantity` ile bir `SalesLineItem` yaratır. Yaratılan nesne, parametre olarak aldığı `spec`'i kendi özelliğine atar. Artık `SalesLineItem` için bu bir özellik görünürlüğüdür.

Hoca, genellikle olanın bu olduğunu söyledi: Parametre olarak gelen nesneleri değişkenlere atarsınız ve böylece daha fazla bağımlı hâle gelirsiniz. Bunu mümkün olduğunca azaltmaya gayret etmemiz gerekir. Kasa da `spec`'i ayrı bir değişkene almak yerine yalnız parametre bazında tutabilirdi.

> **Ders sonrası not:** Hoca bir yerde "metot parametresini yerel değişkene atarsanız özellik görünürlüğüne dönüşür", bir yerde de "yerel nitelik görünürlüğü" dedi. Kendi tanımlarına göre doğrusu şudur: Parametre metot içinde yerel değişkene atanırsa yerel görünürlük olur. Nesnenin bir özelliğine atanırsa özellik görünürlüğü olur. Örnekteki `SalesLineItem` ikinci duruma uyar.

18. Tasarım Sınıf Diyagramı

Senaryolar gerçekleştirilip etkileşim diyagramları çizildikten sonra bunlarla birlikte yazılım sınıf diyagramı oluşturulur. Bu diyagramda bütün sınıflar, özellikleri, tipleri, metotları, metot parametreleri ve erişim hakları gösterilir. İlişkiler ve bağımlılıklar yönlü çizilir. Kavramsal modelde olmayan ama yazılımla ortaya çıkan yeni bağlantılar da olabilir. Bu diyagram yazılımın statik yapısını gösterir: "Yazılım sınıflarım bunlardır, aralarındaki ilişkiler de böyle olmalıdır."

Hoca, dersin birkaç haftadır kullandığı örnek için diyagramı gösterdi: dükkan, kasa, katalog, satış, ürün tanımı, satış satırı ve ödeme. Sınıflarda artık metotlar da vardır. Gerçekte çok daha fazla metot olur, çünkü bunlar yalnız bir iki senaryodan bulunmuştur. Anlatımında şu bağlantıların nedenlerini söyledi:

- `Register`, `ProductCatalog`'a bakar, çünkü `getSpecification` çağırır.
- `Store`'da `Register` vardır, çünkü onu yaratır.
- `Sale`, ödemenin sahibidir ve onunla ödeme yapılır.
- Her `Sale`'in içinde n tane `SalesLineItem` vardır. Bu satırların listesini tutar ve onları yaratır.

Diyagramın tamamı altyazıda okunmuyor. Aşağıdaki tablo kısa nottan alındı:

Sınıf	Özellikler	Metotlar	Bağlantılar
`Store`	address, name	`addSale()`	`ProductCatalog`'u kullanır (Uses), `Register`'ı barındırır (Houses), tamamlanan `Sale`'lerin kaydını tutar (Logs-completed)
`Register`		`endSale()`, `enterItem()`, `makeNewSale()`, `makePayment()`	`ProductCatalog`'a bakar (Looks-in), `Sale`'i yakalar (Captures)
`ProductCatalog`		`getSpecification()`	1..* `ProductSpecification` içerir
`ProductSpecification`	description, price, itemID		
`Sale`	date, isComplete, time	`becomeComplete()`, `makeLineItem()`, `makePayment()`, `getTotal()`	1..* `SalesLineItem` içerir, bir `Payment` ile ödenir (Paid-by)
`SalesLineItem`	quantity	`getSubtotal()`	Bir `ProductSpecification` tarafından tanımlanır (Describes)
`Payment`	amount		

Ek olarak bağımlılıklar (dependency) da gösterilebilir. Örneğin kasa, katalogdan aldığı ürün tanımını yerel değişkene atıyordu, bu yüzden ona bağımlıdır. Görebildiğiniz bağımlılıkları kesikli çizgiyle göstermekte fayda vardır. Hoca bunun üzerinde çok durmadı.

Özetle, etkileşim diyagramlarıyla yazılımın nasıl işleyerek senaryoları yerine getireceğini, yazılım sınıf diyagramıyla da statik yapıyı gösteririz. Nesneye yönelik yazılım tasarımının ifadesi bu ikisidir.

19. Kodlamaya Geçiş

Sırada kodlama vardır. Sınıf tanımları yazılım sınıf diyagramından, metotlar etkileşim diyagramlarından çıkarılır. Bunlar bizim için birer şablondur.

Günümüzde CASE araçları bu şablonu üretebilir. ArgoUML ya da Visual Paradigm gibi bir araçta diyagramı çizip “Java’nın şu sürümü için kod şablonu oluştur” dediğinizde, listeleri ve döngüleriyle birlikte iskelet çıkar. Geriye metotların içindeki algoritmaları, yani iç mantığı kodlamak kalır.

> **Hocanın vurgusu:** Kodlamaya en az bağımlı sınıftan başlayıp en çok bağımlı sınıfa doğru gidin. Her sınıfı yarattıkça mutlaka birim teste tabi tutun.

Test programını daha önce de yazabilirsiniz. Buna **test güdümlü geliştirme (test driven development)** denir. Test programı, sınıftan nesneler yaratıp olabilecek bütün mesaj çağrılarıyla doğru çalışıp çalışmadığını kontrol eder. Hoca bunun şu an için ileri düzey olduğunu, ayrıntılarını Yazılım Mühendisliği dersinde göreceğinizi söyledi.

Dersin Toparlaması

Geçen hafta işlevsel çözümlemede tasarımın yapı diyagramlarıyla nasıl yapıldığını görmüştük. Bu hafta, sınıflar ve senaryolardan yola çıkarak nesneye yönelik bir yazılımın nasıl tasarlandığını gördük. Tasarımın özü, “kim neden sorumlu olsun?” sorusuna cevap vermektir. Bu kararlar senaryolardan çıkan bilme ve yapma sorumlulukları üzerinden, GRASP’ın dokuz prensibi ışığında verilir: Uzman ve Yaratıcı sorumluluğun doğal sahibini gösterir, Az Bağımlılık ve İyi Uyum tasarımın kalitesini tartar, Denetçi sistem olaylarını karşılar, Çok Şekillilik tipe bağlı davranışları düzenler, Yapay Sınıf, Dolaylılık ve Değişimlerden Korunma ise değişime açık noktaların arasına ölçülü tampon bölgeler koyar.

Kararların sonucu, dinamik yapı için etkileşim diyagramlarıyla, statik yapı için tasarım sınıf diyagramıyla ifade edilir. Kodlama da bu iki diyagramdan, en az bağımlı sınıftan başlayarak yürütülür. Sonraki ders, tasarımın son adımı olan veri modelleme ve veri tabanı tasarımına odaklanacak.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.